

Deep Learning from Production to Deployment

Task 6: Creating a Streamlit User Interface

Student Name: Sakshi Vijay Sataye

Roll No: 30

Class: Msc DSBDA Part 2

Subject: Deep Learning from Production to Deployment

Github Repository Link: https://github.com/saakshi5/Deep_Learning_Task6_Creating-a-Streamlit-User-Interface.git

1. Introduction

This project demonstrates the development of a Streamlit-based interactive web application for an E-Commerce Sales Analysis system. The application integrates data analytics, visualization, and machine learning to provide real-time business insights from retail transaction data. Users can dynamically filter sales data, visualize trends, and perform customer segmentation using K-Means clustering through a user-friendly dashboard.

2. Objective

- Design an interactive Streamlit web interface.
- Create user input forms and sidebar widgets.
- Connect the UI with prediction and analytical logic.
- Display dynamic charts and KPI metrics.
- Improve usability using layouts, columns, expanders, and filters.
- Develop a professional dashboard for business analysis.

3. Reference Dataset

The task uses the Online Retail E-Commerce Dataset (Kaggle).

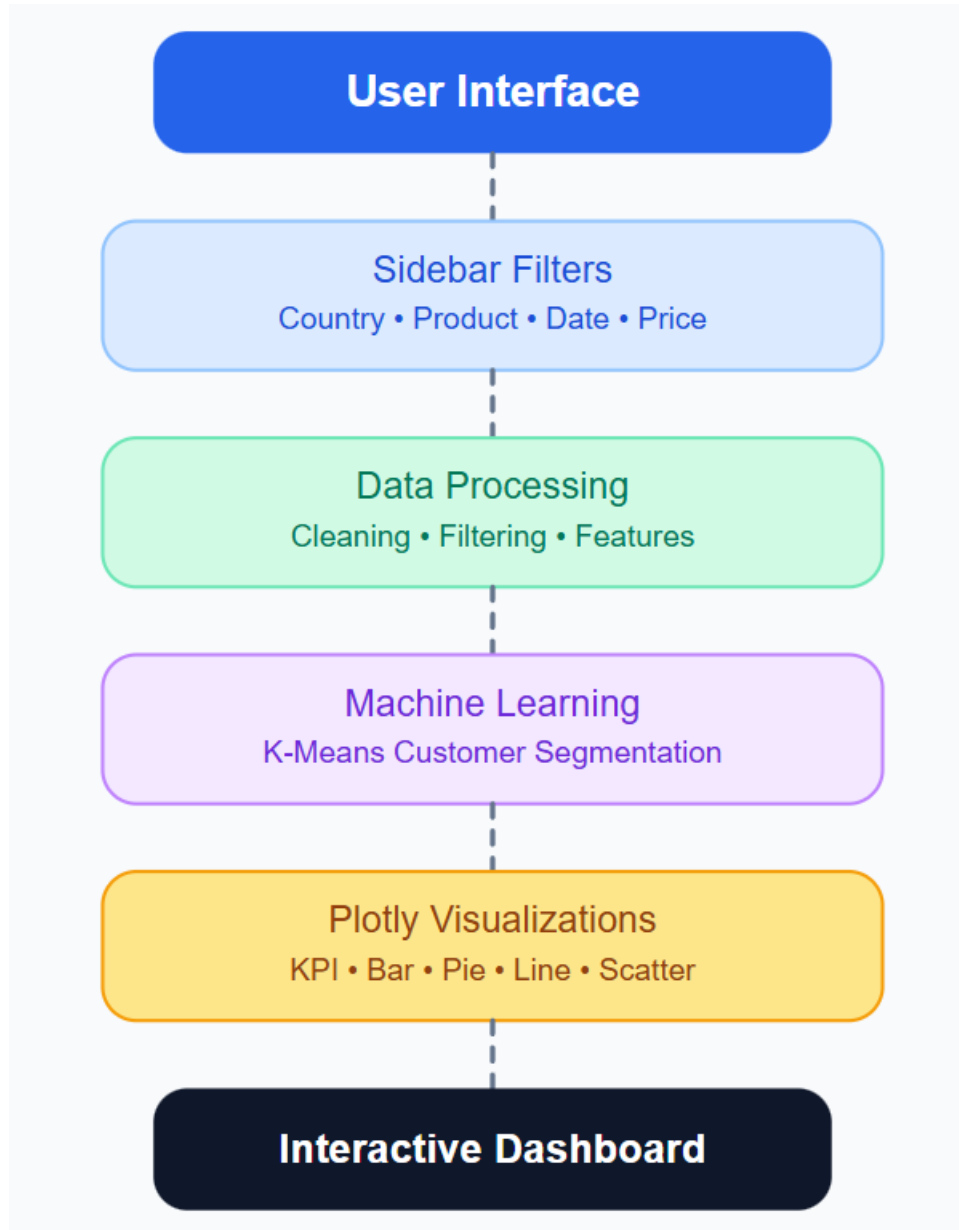
Feature	Description
Dataset	Online Retail Dataset
Records	Retail Transactions
Domain	E-Commerce
Format	Excel (.xlsx)
Key Fields	Invoice, CustomerID, Country, Product, Quantity, Price

The dataset contains customer purchases, product descriptions, invoice dates, prices, quantities, and country information used for business analytics.

4. Tools & Technologies

Tool	Purpose
Python	Programming Language
Streamlit	User Interface Development
Pandas	Data Processing
NumPy	Numerical Operations
Plotly Express	Interactive Charts
Scikit-learn	K-Means Clustering
VS Code	Development Environment
Excel Dataset	Data Source

5. Streamlit Application Architecture



Explanation

The Streamlit application loads the dataset, applies user-selected filters from the sidebar, processes the data, performs customer segmentation using K-Means, and displays interactive visualizations with real-time updates.

6. User Interface Design

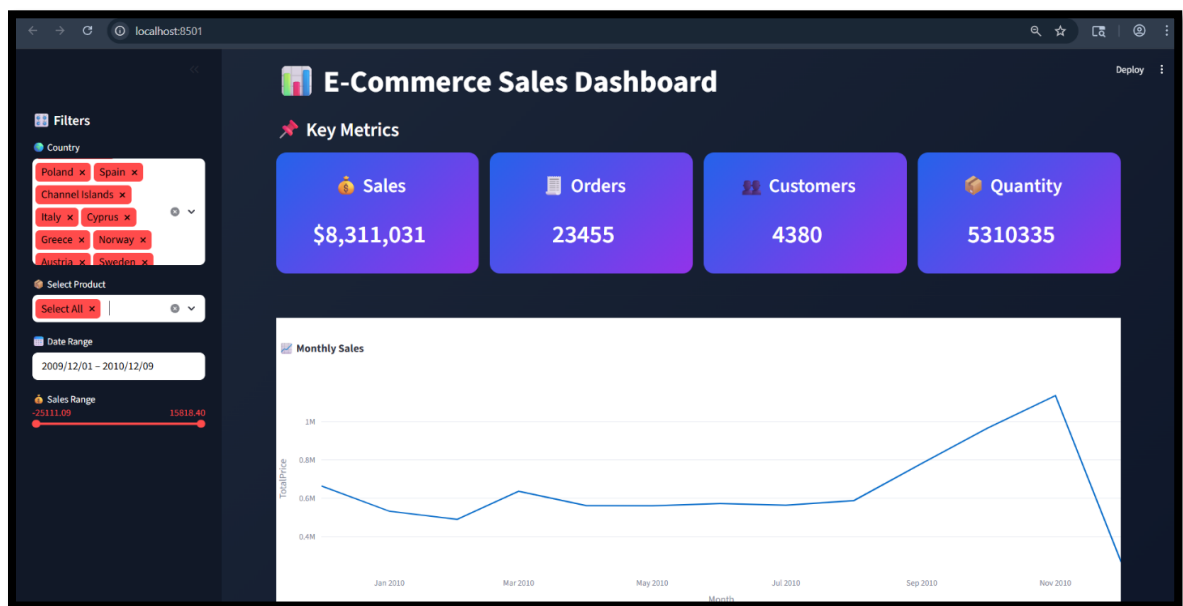
The dashboard is designed with a modern dark theme and interactive components.

6.1: E-Commerce Sales Dashboard Overview & Key Performance Metrics



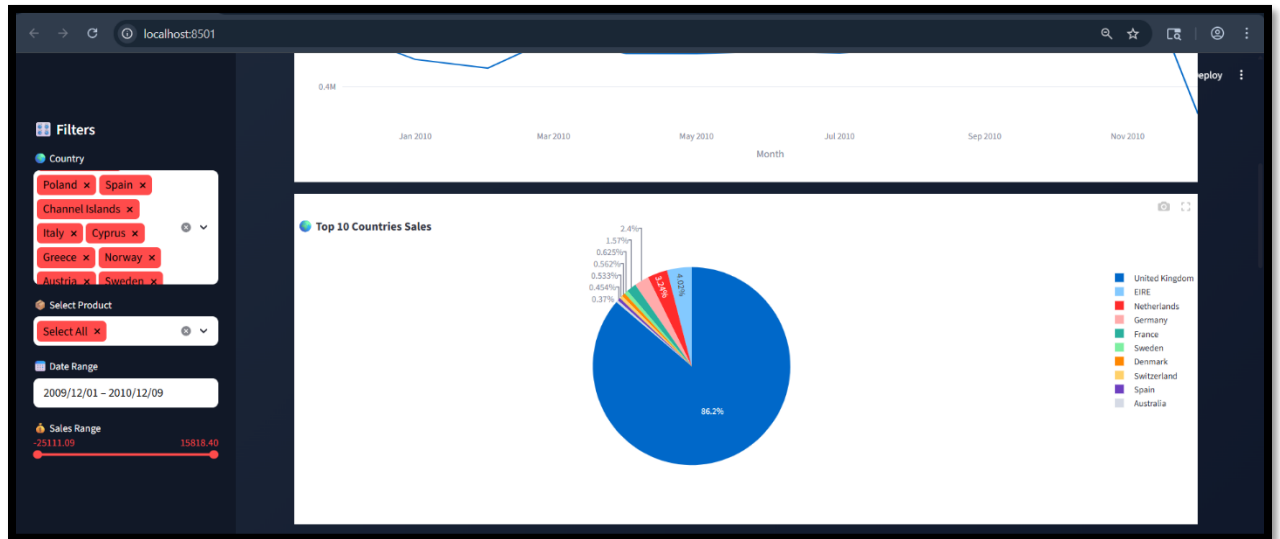
Displays the main dashboard interface highlighting four critical business KPIs: total sales (\$8,311,031), total orders (23,455), unique customers (4,380), and total quantity sold (5,310,335).

6.2: Dashboard Sidebar Filters and Main KPI Layout



Illustrates the active multi-select filters in the sidebar (Country, Product, Date Range, Sales Range) dynamically updating the main metrics on the dashboard.

6.3: Top 10 Countries Sales Share Breakdown



Shows a Plotly pie chart visualizing the "Top 10 Countries Sales," revealing that the United Kingdom accounts for the vast majority of sales at 86.2%.

6.4: Monthly Country Sales Trends Comparison



Features a stacked bar chart mapping "Monthly Country Sales" to compare transaction volume across different regions month-by-month throughout 2010.

6.5: Aggregate Sales by Country and Top 5 Customers Distribution



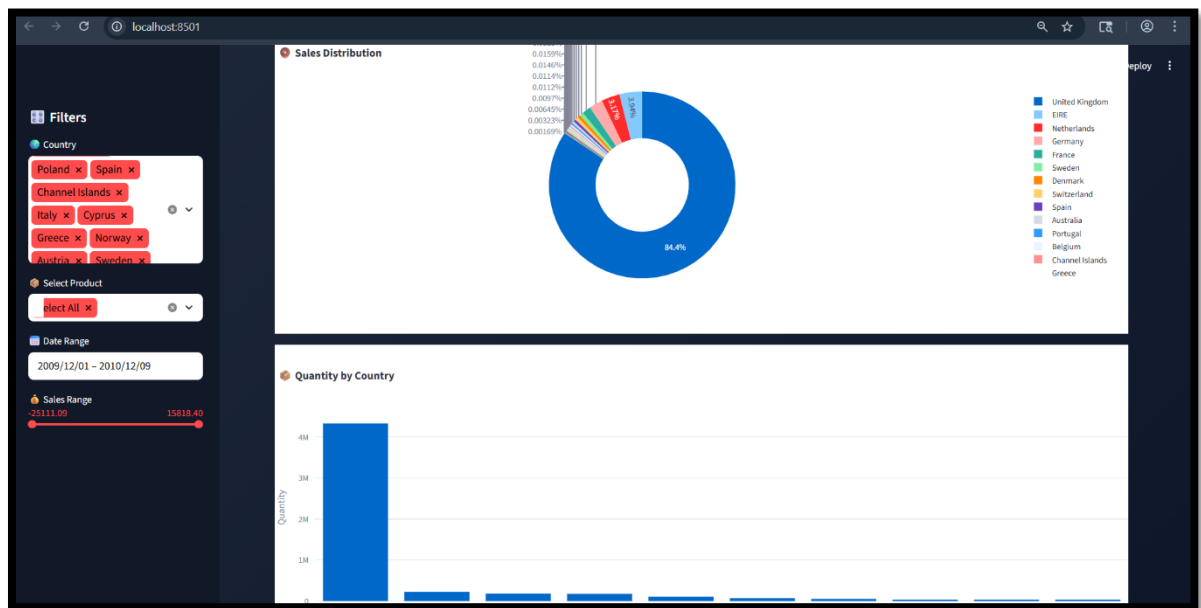
Displays a bar chart comparing total sales revenue across countries alongside a pie chart showcasing the distribution of the "Top 5 Customers."

6.6: Top Customer Analysis and Monthly Orders Volume



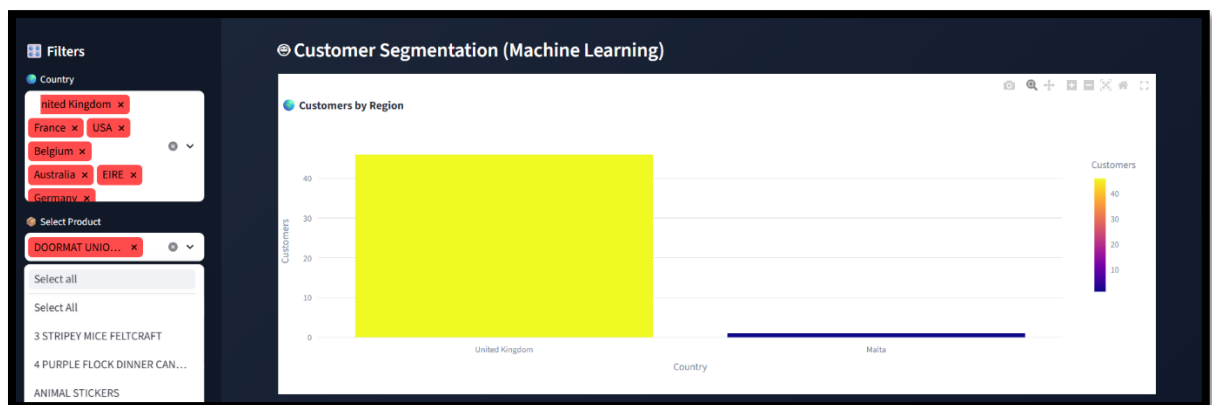
Combines a customer allocation pie chart with a colorful "Monthly Orders" bar chart tracking transaction frequencies across the year.

6.7: Total Sales Distribution and Quantity by Country Insights



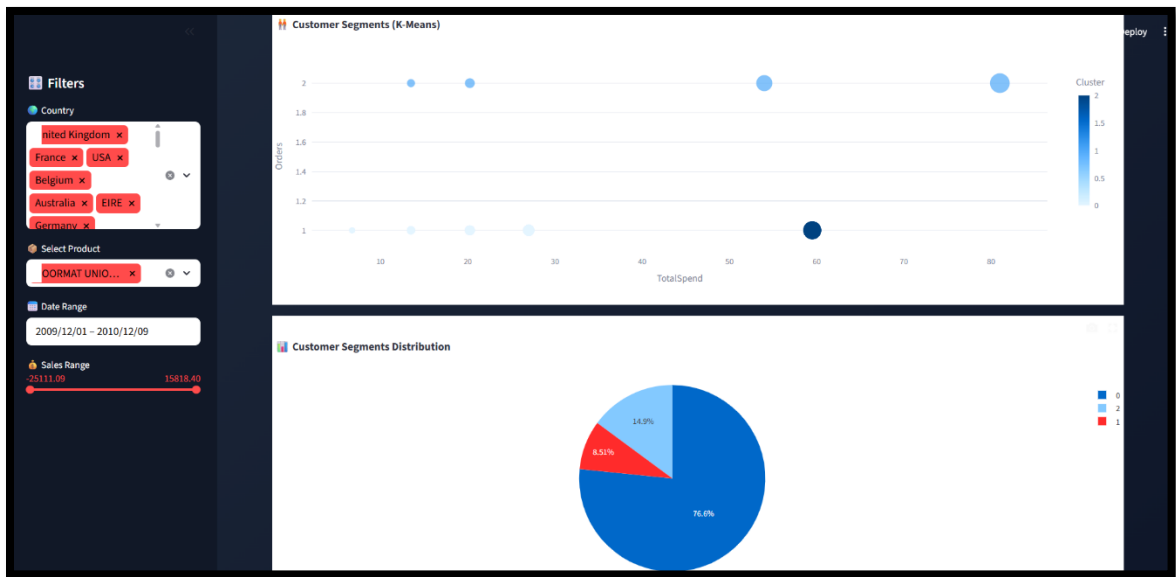
Presents a localized "Sales Distribution" donut chart next to a "Quantity by Country" bar chart highlighting product distribution volume.

6.8: Customer Segment Counts by Region



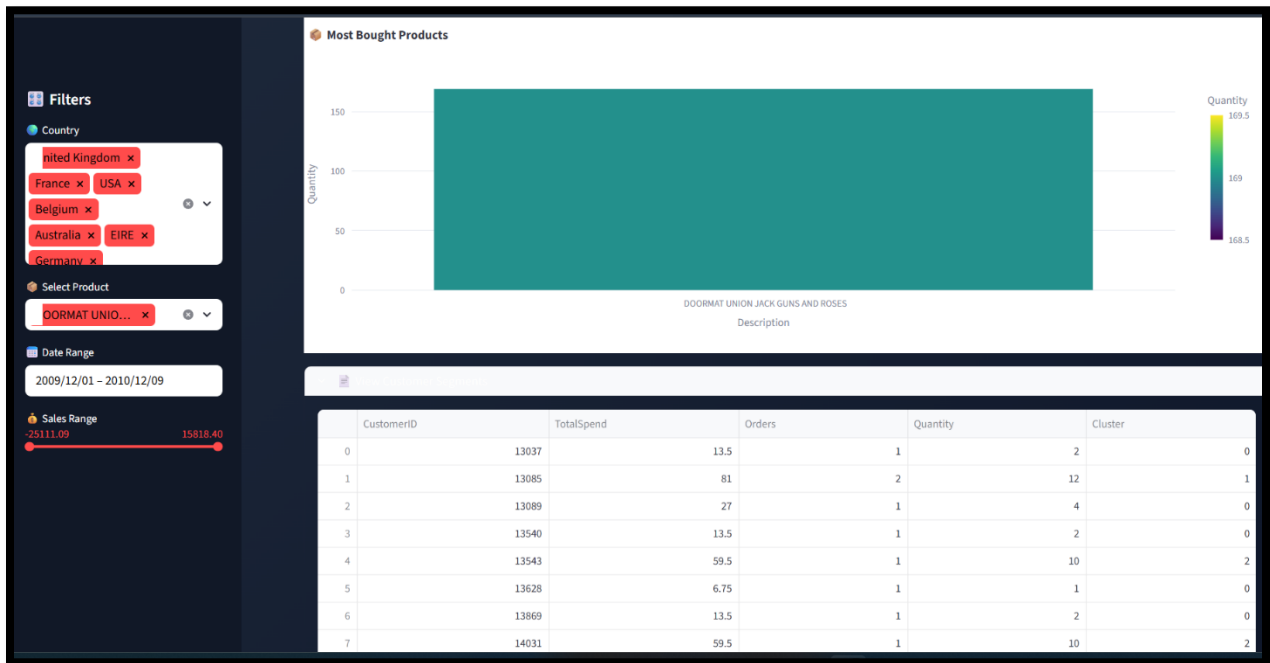
Shows the "Customers by Region" bar chart within the Machine Learning section, highlighting a massive concentration of customers in the United Kingdom compared to Malta.

6.9: K-Means Customer Segments Scatter Plot and Cluster Distribution



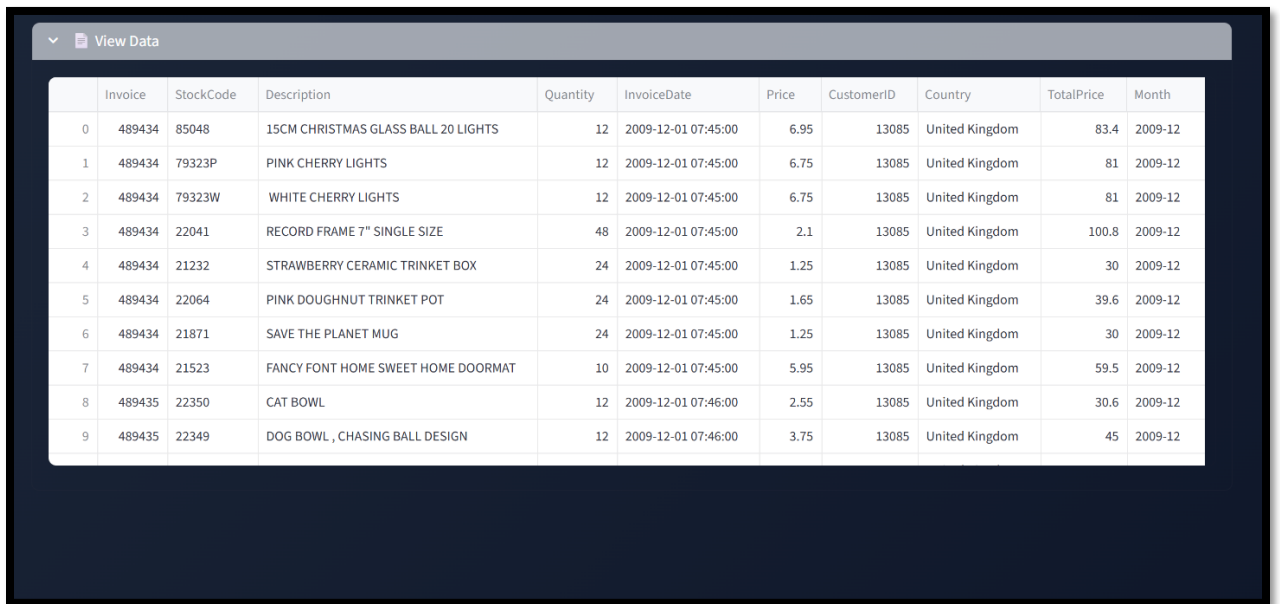
Visualizes the K-Means customer clustering results through a scatter plot of orders versus total spend alongside a pie chart showing the distribution of the three distinct clusters.

6.10: Most Bought Products Bar Chart and Segmented Customer Data Table



Illustrates the "Most Bought Products" bar chart positioned directly above an interactive, segmented data table detailing individual customer cluster assignments.

6.11: Expanded View of the Raw Transaction Dataset



The screenshot shows a dark-themed interface with a 'View Data' expander widget. The widget is expanded, revealing a table with 11 columns: Invoice, StockCode, Description, Quantity, InvoiceDate, Price, CustomerID, Country, TotalPrice, and Month. The table contains 10 rows of data, each representing a transaction. The data is as follows:

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	CustomerID	Country	TotalPrice	Month
0	489434	85048	15CM CHRISTMAS GLASS BALL 20 LIGHTS	12	2009-12-01 07:45:00	6.95	13085	United Kingdom	83.4	2009-12
1	489434	79323P	PINK CHERRY LIGHTS	12	2009-12-01 07:45:00	6.75	13085	United Kingdom	81	2009-12
2	489434	79323W	WHITE CHERRY LIGHTS	12	2009-12-01 07:45:00	6.75	13085	United Kingdom	81	2009-12
3	489434	22041	RECORD FRAME 7" SINGLE SIZE	48	2009-12-01 07:45:00	2.1	13085	United Kingdom	100.8	2009-12
4	489434	21232	STRAWBERRY CERAMIC TRINKET BOX	24	2009-12-01 07:45:00	1.25	13085	United Kingdom	30	2009-12
5	489434	22064	PINK DOUGHNUT TRINKET POT	24	2009-12-01 07:45:00	1.65	13085	United Kingdom	39.6	2009-12
6	489434	21871	SAVE THE PLANET MUG	24	2009-12-01 07:45:00	1.25	13085	United Kingdom	30	2009-12
7	489434	21523	FANCY FONT HOME SWEET HOME DOORMAT	10	2009-12-01 07:45:00	5.95	13085	United Kingdom	59.5	2009-12
8	489435	22350	CAT BOWL	12	2009-12-01 07:46:00	2.55	13085	United Kingdom	30.6	2009-12
9	489435	22349	DOG BOWL , CHASING BALL DESIGN	12	2009-12-01 07:46:00	3.75	13085	United Kingdom	45	2009-12

Demonstrates the collapsible "View Data" expander widget, showcasing the raw cleaned transaction dataset columns like StockCode, InvoiceDate, and TotalPrice.

7. Implementation

Step 1: Data Utility Processing (utils.py)

- Locate Dataset: Finds the correct local directory path for the online_retail.xlsx data source file.
- Format Features: Cleans column spaces and casts StockCode and Invoice entries into uniform text strings.
- Calculate Totals: Drops empty values, converts dates, and calculates TotalPrice by multiplying quantity by unit price.

Code:

```
import pandas as pd

import os

def load_data():

    base_path = os.path.dirname(os.path.dirname(__file__))
```

```
file_path = os.path.join(base_path, 'data', 'online_retail.xlsx')
df = pd.read_excel(file_path)
```

```
# Clean column names
```

```
df.columns = df.columns.str.strip().str.replace(' ', '')
```

```
# 🔥 FIX ALL PROBLEMATIC COLUMNS
```

```
df['StockCode'] = df['StockCode'].astype(str)
```

```
# Handle both possible names
```

```
if 'Invoice' in df.columns:
```

```
    df['Invoice'] = df['Invoice'].astype(str)
```

```
elif 'Invoice' in df.columns:
```

```
    df['Invoice'] = df['Invoice'].astype(str)
```

```
# Core processing
```

```
df.dropna(inplace=True)
```

```
df['TotalPrice'] = df['Quantity'] * df['Price']
```

```
df['InvoiceDate'] = pd.to_datetime(df['InvoiceDate'])
```

```
df['Month'] = df['InvoiceDate'].dt.month
```

```
return df
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
# Load data
```

```
df = pd.read_excel('../data/online_retail.xlsx')
```

```
# Basic info
```

```
print(df.head())

print(df.info())

# Handle missing values

df.dropna(inplace=True)

# Create TotalPrice column

df['TotalPrice'] = df['Quantity'] * df['Price']

# Convert InvoiceDate

df['InvoiceDate'] = pd.to_datetime(df['InvoiceDate'])

# Extract Month

df['Month'] = df['InvoiceDate'].dt.month

# Top Countries

top_countries = df.groupby('Country')['TotalPrice'].sum().sort_values(ascending=False)

# Plot

plt.figure(figsize=(10,5))

top_countries.head(10).plot(kind='bar')

plt.title("Top Countries by Sales")

plt.show()

# Monthly Sales

monthly_sales = df.groupby('Month')['TotalPrice'].sum()

plt.figure(figsize=(8,4))

monthly_sales.plot(marker='o')

plt.title("Monthly Sales Trend")

plt.show()
```

Step 2: Machine Learning Clustering (ml.py)

- Clean Outliers: Filters out negative purchase numbers and isolates rows containing valid customer IDs.
- Aggregate Actions: Groups data by user to calculate their total spend, total unique orders, and total item counts.
- Scale Data: normalizes metrics using StandardScaler to ensure balanced numerical input weight.
- Segment Customers: Fits a 3-cluster KMeans model to divide shoppers into Low, Medium, and High-Value (VIP) tiers.

Code:

```
import pandas as pd

from sklearn.cluster import KMeans

from sklearn.preprocessing import StandardScaler

def customer_segmentation(df):

    # ===== CLEAN DATA =====

    # Remove negative quantities (VERY IMPORTANT)
    df = df[df['Quantity'] > 0]

    # Remove missing CustomerID
    df = df.dropna(subset=['CustomerID'])

    # ===== CREATE CUSTOMER DATA =====

    customer_df = df.groupby('CustomerID').agg({

        'TotalPrice': 'sum',

        'Invoice': 'nunique',

        'Quantity': 'sum'

    }).reset_index()
```

```

customer_df.columns = ['CustomerID', 'TotalSpend', 'Orders', 'Quantity']

# Extra safety cleanup
customer_df = customer_df[customer_df['Quantity'] > 0]

# ===== SCALING =====

scaler = StandardScaler()

scaled_data = scaler.fit_transform(
    customer_df[['TotalSpend', 'Orders', 'Quantity']]
)

# ===== K-MEANS MODEL =====

kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
customer_df['Cluster'] = kmeans.fit_predict(scaled_data)

return customer_df

```

Step 3: Exploratory Data Analysis (eda_analysis.ipynb)

- **Inspect Structure:** Imports standard analytical libraries (pandas, matplotlib, seaborn) to view basic dataset configurations.
- **Identify Markets:** Groups global records by country to reveal and plot the top 10 revenue-generating locations.
- **Map Fluctuations:** Aggregates transaction data by month to plot operational sales trends across the year.

Code:

```

countries = st.sidebar.multiselect(
    "Country",
    df["Country"].unique()
)

date_range = st.sidebar.date_input(
    "Date Range"
)

```

```
)  
price = st.sidebar.slider(  
    "Sales Range",  
    0.0,  
    5000.0  
)
```

Step 4: Display KPI Metrics

```
c1, c2, c3, c4 = st.columns(4)  
c1.metric("Sales", total_sales)  
c2.metric("Orders", total_orders)  
c3.metric("Customers", customers)  
c4.metric("Quantity", quantity)
```

Step 4: Web Application Dashboard (app.py)

- **Style Interface:** Configures a wide Streamlit window using customized dark theme CSS stylesheets.
- **Gather Controls:** Adds multi-select sidebar widgets for country and product picking, plus sliders for date and price parameters.
- **Compute KPIs:** Uses dynamic grid columns to display overall metrics for Sales, Orders, Customers, and Quantities.
- **Render Graphics:** Displays interactive Plotly charts tracking monthly sales, regional shares, customer cluster scatter plots, and top products.
- **Reveal Data:** Embeds clean user-facing data containers using collapsible expander drawers to inspect individual rows.

Code:

```
from ml import customer_segmentation  
  
import streamlit as st  
  
import pandas as pd  
  
import plotly.express as px
```

```

import os

# ===== PAGE CONFIG =====

st.set_page_config(page_title="Ecommerce Dashboard", layout="wide")

# ===== UI =====

st.markdown("""

<style>

/* ===== SIDEBAR TOGGLE BUTTON ===== */

/* FORCE sidebar toggle visibility */

button[data-testid="baseButton-header"] svg {

    fill: white !important;

    color: white !important;

}

/* fallback for newer versions */

[data-testid="stAppViewContainer"] header svg {

    color: white !important;

}

/* Ensure visibility on dark header */

header[data-testid="stHeader"] {

    background: transparent !important;

}

[data-testid="stAppViewContainer"] {

    background: linear-gradient(135deg, #1e293b, #0f172a);

    color: white;

}

```

```
[data-testid="stSidebar"] {  
    background: #111827;  
}  
  
/* Sidebar text (keep white) */  
[data-testid="stSidebar"] label,  
[data-testid="stSidebar"] h1,  
[data-testid="stSidebar"] h2,  
[data-testid="stSidebar"] h3 {  
    color: white !important;  
}  
  
/* ONLY input fields text black */  
[data-testid="stSidebar"] input,  
[data-testid="stSidebar"] textarea {  
    color: black !important;  
    background-color: white !important;  
}  
  
  
/* Multiselect selected values (placeholder area) */  
[data-baseweb="select"] span {  
    color: black !important;  
}  
  
/* Dropdown menu options */  
[data-baseweb="menu"] div {  
    color: black !important;  
}  
  
/* Fix calendar popup */
```

```
[data-baseweb="input"] input {
    color: black !important;
}

/* Remove red border / error highlight */

[data-baseweb="select"] div {
    border: none !important;
    box-shadow: none !important;
}

/* Main headings */

h1, h2, h3 {
    color: #facc15;
}

/* KPI */

.kpi {
    background: linear-gradient(135deg, #2563eb, #9333ea);
    padding: 20px;
    border-radius: 15px;
    text-align: center;
    color: white;
}

/* Charts */

.chart-box {
    background: #1f2937;
    padding: 15px;
    border-radius: 15px;
    margin-bottom: 20px;
```

```

}

    /* ML section box */
.ml-box {
    background: #0f172a;
    padding: 20px;
    border-radius: 15px;
    margin-top: 20px;
    border: 1px solid #334155;
}
</style>

"", unsafe_allow_html=True)

# ===== TITLE =====

st.title("🇮🇹 E-Commerce Sales Dashboard")

# ===== LOAD DATA =====

@st.cache_data
def load_data():
    base_path = os.path.dirname(os.path.dirname(__file__))
    file_path = os.path.join(base_path, 'data', 'online_retail.xlsx')
    df = pd.read_excel(file_path)
    df.columns = df.columns.str.strip().str.replace(' ', '')
    df['InvoiceDate'] = pd.to_datetime(df['InvoiceDate'], errors='coerce')
    df['Quantity'] = pd.to_numeric(df['Quantity'], errors='coerce')
    df['Price'] = pd.to_numeric(df['Price'], errors='coerce')
    df['TotalPrice'] = df['Quantity'] * df['Price']

```

```

df['Month'] = df['InvoiceDate'].dt.to_period('M').astype(str)

df.dropna(inplace=True)

return df


# ===== SIDEBAR =====

df = load_data()

st.sidebar.markdown("## 🛠️ Filters")

countries = st.sidebar.multiselect(

    "🌍 Country",

    df['Country'].unique(),

    default=df['Country'].unique()

)


# ===== PRODUCT FILTER =====

product_list = sorted(df['Description'].dropna().unique().tolist())

product_options = ["Select All"] + product_list

selected_products = st.sidebar.multiselect(

    "📦 Select Product",

    options=product_options,

    default=["Select All"]

)

if "Select All" in selected_products:

    products = product_list

else:

    products = selected_products

if not products:

```

```

products = df['Description'].unique()

min_date = df['InvoiceDate'].min().date()
max_date = df['InvoiceDate'].max().date()

date_range = st.sidebar.date_input("📅 Date Range", (min_date, max_date))
start_date, end_date = date_range

price_range = st.sidebar.slider(
    "💰 Sales Range",
    float(df['TotalPrice'].min()),
    float(df['TotalPrice'].max()),
    (float(df['TotalPrice'].min()), float(df['TotalPrice'].max()))
)

# ===== FILTER =====

filtered_df = df[
    (df['Country'].isin(countries)) &
    (df['Description'].isin(products)) & # ✅ ADD THIS
    (df['InvoiceDate'] >= pd.to_datetime(start_date)) &
    (df['InvoiceDate'] <= pd.to_datetime(end_date)) &
    (df['TotalPrice'].between(price_range[0], price_range[1]))
]

if filtered_df.empty:
    st.warning("No data → showing full dataset")
    filtered_df = df.copy()

# ===== KPI =====

st.subheader("📌 Key Metrics")

```

```

c1, c2, c3, c4 = st.columns(4)

c1.markdown(f"<div class='kpi'><h3> 💰  
Sales</h3><h2>${filtered_df['TotalPrice'].sum():.0f}</h2></div>", unsafe_allow_html=True)

c2.markdown(f"<div class='kpi'><h3> 📄  
Orders</h3><h2>{filtered_df['Invoice'].nunique()}</h2></div>", unsafe_allow_html=True)

c3.markdown(f"<div class='kpi'><h3> 👤  
Customers</h3><h2>{filtered_df['CustomerID'].nunique()}</h2></div>",  
unsafe_allow_html=True)

c4.markdown(f"<div class='kpi'><h3> 📦  
Quantity</h3><h2>{int(filtered_df['Quantity'].sum())}</h2></div>",  
unsafe_allow_html=True)

st.markdown("---")

# ===== CHARTS =====

country_sales = filtered_df.groupby('Country')['TotalPrice'].sum().reset_index()

# 📊 Monthly Sales

monthly = filtered_df.groupby('Month')['TotalPrice'].sum().reset_index()

st.plotly_chart(px.line(monthly, x='Month', y='TotalPrice', title="📊 Monthly Sales"),  
width="stretch")

# 🥧 Pie Top Countries

top_country = country_sales.sort_values(by='TotalPrice', ascending=False).head(10)

st.plotly_chart(px.pie(top_country, values='TotalPrice', names='Country', title="🌍 Top 10  
Countries Sales"), width="stretch")

# 📊 Stacked Bar

stacked = filtered_df.groupby(['Month', 'Country'])['TotalPrice'].sum().reset_index()

st.plotly_chart(px.bar(stacked, x='Month', y='TotalPrice', color='Country', barmode='stack',  
title="📊 Monthly Country Sales"), width="stretch")

```

🇮🇹 Bar: Top Countries Sales

```
st.plotly_chart(px.bar(top_country, x='Country', y='TotalPrice', title="🇮🇹 Sales by Country",  
color='TotalPrice'), width="stretch")
```

🥧 Pie Top Customers

```
top_cust =  
filtered_df.groupby('CustomerID')['TotalPrice'].sum().reset_index().sort_values(by='TotalPri  
ce', ascending=False).head(5)
```

```
st.plotly_chart(px.pie(top_cust, values='TotalPrice', names='CustomerID', title="👑 Top 5  
Customers"), width="stretch")
```

🇮🇹 Bar: Monthly Orders

```
monthly_orders = (  
    filtered_df.groupby('Month')['Invoice']  
    .nunique()  
    .reset_index()  
    .sort_values(by='Invoice') # 👉 ascending order  
)
```

```
fig = px.bar(  
    monthly_orders,  
    x='Month',  
    y='Invoice',  
    title="🇮🇹 Monthly Orders",  
    color='Invoice', # 👉 Adds color
```

```

    color_continuous_scale='viridis' # 👉 Nice gradient
)

st.plotly_chart(fig, width="stretch")

# 🍷 Donut Chart

st.plotly_chart(px.pie(country_sales, values='TotalPrice', names='Country', hole=0.5,
title="🍷 Sales Distribution"), width="stretch")

# 📊 Bar: Quantity by Country

qty_country =
filtered_df.groupby('Country')['Quantity'].sum().reset_index().sort_values(by='Quantity',
ascending=False).head(10)

st.plotly_chart(px.bar(qty_country, x='Country', y='Quantity', title="📦 Quantity by
Country"), width="stretch")

# ===== ML SECTION =====

from ml import customer_segmentation

st.markdown("---")

st.subheader("🤖 Customer Segmentation (Machine Learning)")

# Run ML model

customer_df = customer_segmentation(filtered_df)

# 🔥 ADD THIS HERE (IMPORTANT)

customer_df['Cluster'] = customer_df['Cluster'].map({

    0: "🟢 Low Value Customers",

    1: "🟡 Medium Value Customers",

    2: "🔴 High Value Customers (VIP)"

})

```

```
# Remove nulls just in case
```

```
customer_df = customer_df.dropna()
```

```
region_df = filtered_df.groupby('Country')['CustomerID'].nunique().reset_index()
```

```
region_df.columns = ['Country', 'Customers']
```

```
fig_region = px.bar(
```

```
    region_df.sort_values(by='Customers', ascending=False).head(10),
```

```
    x='Country',
```

```
    y='Customers',
```

```
    title="🌍 Customers by Region",
```

```
    color='Customers',
```

```
    color_continuous_scale='plasma'
```

```
)
```

```
st.plotly_chart(fig_region, width="stretch")
```

```
# ===== SCATTER PLOT =====
```

```
fig_ml1 = px.scatter(
```

```
    customer_df,
```

```
    x='TotalSpend',
```

```
    y='Orders',
```

```
    color='Cluster',
```

```
    size='Quantity',
```

```
    title="👤👤 Customer Segments (K-Means)"
```

```

)

st.plotly_chart(fig_ml1, width="stretch")

# ===== PIE CHART =====

cluster_dist = customer_df['Cluster'].value_counts().reset_index()

cluster_dist.columns = ['Cluster', 'Count']

fig_ml2 = px.pie(

    cluster_dist,

    values='Count',

    names='Cluster',

    title="🇮🇹 Customer Segments Distribution"

)

st.plotly_chart(fig_ml2, width="stretch")

top_products = filtered_df.groupby('Description')['Quantity'].sum().reset_index()


fig_products = px.bar(

    top_products.sort_values(by='Quantity', ascending=False).head(10),

    x='Description',

    y='Quantity',

    title="📦 Most Bought Products",

    color='Quantity',

    color_continuous_scale='viridis'

)

st.plotly_chart(fig_products, width="stretch")

# ===== DATA TABLE =====

with st.expander("📄 View Customer Segments"):

    st.dataframe(customer_df)

```

```
# ===== DATA =====
```

```
with st.expander("📄 View Data"):
```

```
    st.dataframe(filtered_df.head(50))
```

8. Results

The Streamlit application successfully provides an interactive interface for exploring e-commerce sales data. Users can filter records, visualize sales trends, analyze customer behavior, and view machine learning-based customer segmentation in real time. The dashboard offers fast, responsive, and user-friendly business analytics without requiring SQL queries or manual data processing.

9. Experimental Observations

- Streamlit provides a responsive and interactive web interface.
- Sidebar widgets dynamically update all visualizations.
- Plotly charts improve user interaction through zooming and hovering.
- KPI cards provide instant business summaries.
- K-Means clustering successfully segments customers into three value groups.
- Expanders help organize detailed tables without cluttering the dashboard.
- The application supports real-time filtering and analysis.

10. Conclusion

The E-Commerce Sales Analysis project successfully demonstrates the creation of a professional Streamlit User Interface integrated with data analytics and machine learning. The dashboard enables users to interactively explore sales data, filter records using widgets, visualize business trends through Plotly charts, and perform customer segmentation using K-Means clustering. This project highlights how Streamlit can transform analytical models into practical, user-friendly web applications for business intelligence and decision support.